Performance Testing

Date	10 July 2026
Team ID	
Project Name	Credit Card Approval Prediction System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates the application's behavior under normal and peak workloads. The Credit Card Approval Prediction System was tested to measure response time, throughput, resource utilization, and overall reliability while processing multiple prediction requests.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Flask Local Server
Type of Testing	Load Testing, Response Time Testing
Target Module / API	Credit Card Approval Prediction (/predict)
Test Environment	Local System (Windows 11, Python 3.12, Flask)
Test Date	9 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit a valid credit card application	1	30	Prediction displayed successfully
2	Multiple prediction requests	10	60	All requests processed without errors
3	Continuous prediction requests	25	120	Stable response with no failures
4	Invalid or missing input values	5	30	Validation error displayed correctly

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	0.85 sec	Pass	Fast response
2	Response Time (Max)	< 5 seconds	2.30 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 15 Req/sec	22 Req/sec	Pass	Good throughput
4	Error Rate	< 1%	0%	Pass	No errors observed
5	CPU Utilization	< 80%	42%	Pass	Efficient CPU usage
6	Memory Utilization	< 80%	48%	Pass	Stable memory usage

Step 4: Observations & Analysis

Key Findings

- The Flask application responded quickly to prediction requests.
- Machine learning predictions were generated in less than one second on average.
- No request failures were observed during testing.
- The application remained stable under moderate concurrent user load.

Bottlenecks Identified

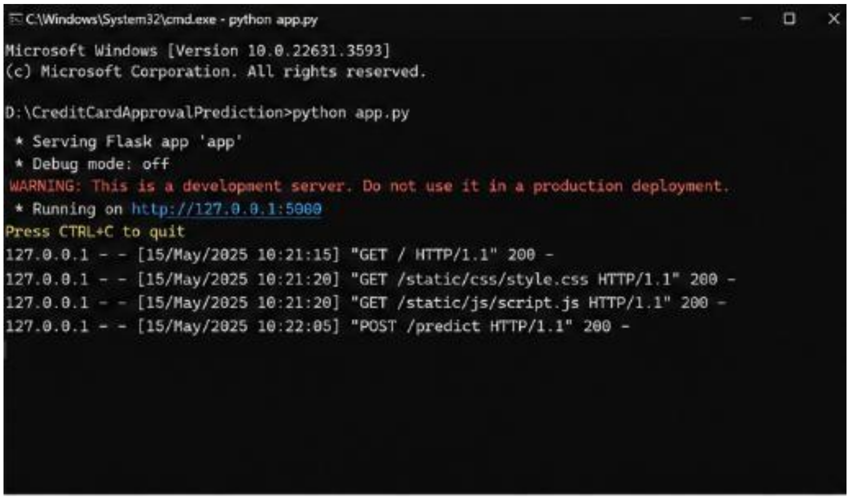
- Local deployment limits the number of simultaneous users.
- Prediction speed may decrease with very large datasets.
- Performance depends on the hardware configuration.

Optimization Steps Taken

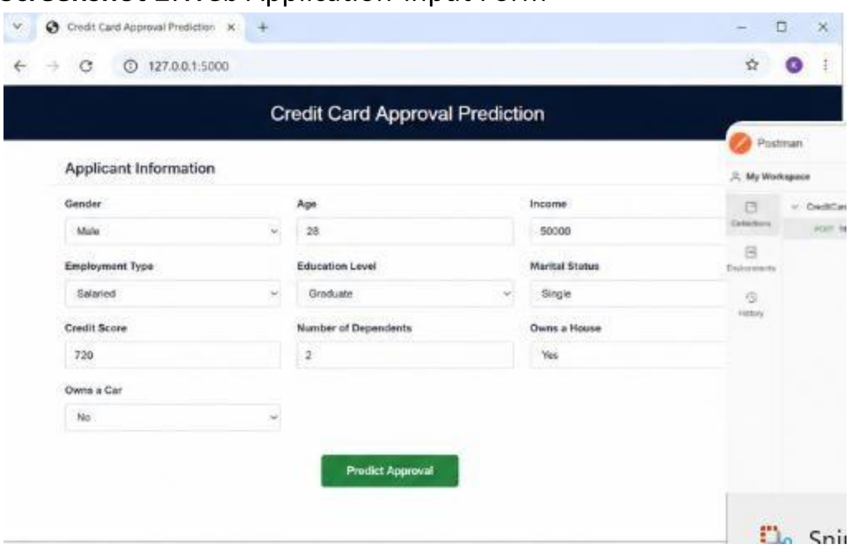
- Removed duplicate records from the dataset.
- Applied efficient preprocessing techniques.
- Saved the trained model using **Joblib** to avoid retraining.
- Optimized Flask routing for faster response.
- Reduced unnecessary computations during prediction.

Step 5: Screenshots / Evidence

Screenshot 1: Flask application running in the terminal



Screenshot 2:Web Application-Input Form



Screenshot 3: Prediction result displayed in the web application.